



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: HERNÁNDEZ FIERROS SONIA

N° de Cuenta: 319194359

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 10 de octubre del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Se modificaron las coordenadas del holocrón que venía en el main. El modelo tiene 16 puntos, pero cada uno pertenece a 3 o 4 caras y cada cara ocupa una zona distinta de la imagen. Como un vértice solo guarda un par de coordenadas (S,T), se repitieron los vértices por cara, igual que en el dado: de 16 pasaron a 56. La imagen se dividió en una cuadrícula de 4×4, con casillas de 0.25 y un margen de 0.01, y a cada cara le tocó una casilla. La textura sigue esa cuadrícula, la textura se pintó en GIMP sobre una plantilla con la figura de cada cara.

```
286 void CrearHolocron()
287 {
288     //indices: cada cara usa SUS vertices (ver el arreglo de
289     unsigned int holocron_indices[] = {
290         // cara 1: cuadrado de arriba (+Y)
291         2, 0, 1,
292         2, 3, 0,
293         // cara 2: cuadrado de enfrente (+Z)
294         4, 6, 7,
295         4, 5, 6,
296         // cara 3: cuadrado derecho (+X)
297         8, 10, 11,
298         8, 9, 10,
299         // cara 4: cuadrado de atras (-Z)
300         12, 14, 15,
301         12, 13, 14,
302         // cara 5: cuadrado izquierdo (-X)
303         19, 17, 18,
304         19, 16, 17,
305         // cara 6: cuadrado de abajo (-Y), trae 8 vertices
306         25, 27, 21,
307         21, 22, 23,
308         23, 24, 25,
309         25, 26, 27,
310         27, 20, 21,
311         21, 23, 25,
312         // cara 7: triangulo arriba-enfrente-derecha
313         30, 28, 29,
```

Función CrearHolocron y se muestran los índices de las primeras caras.

```

341     GLfloat holocron_vertices[] = {
342         // front
343         //x          y          z          S          T          NX          NY          NZ
344         // cara 1: cuadrado de arriba (+Y) -> casilla columna 1, renglon 1 (contando desde arriba a la izquierda
345         0.051152f, 2.399781f, 2.545043f, 0.01f, 0.76f, 0.01f, -1.0f, -0.01f, //0 (antes era el 3)
346         2.521206f, 2.448241f, 0.063388f, 0.24f, 0.76f, 0.01f, -1.0f, -0.01f, //1 (antes era el 14
347         0.049649f, 2.440128f, -2.417217f, 0.24f, 0.99f, 0.01f, -1.0f, -0.01f, //2 (antes era el 2)
348         -2.451372f, 2.421759f, 0.077538f, 0.01f, 0.99f, 0.01f, -1.0f, -0.01f, //3 (antes era el 15
349         // cara 2: cuadrado de enfrente (+Z) -> casilla columna 2, renglon 1 (contando desde arriba a la izquier
350         -0.140696f, -2.601934f, 2.558561f, 0.26f, 0.76f, 0.0f, 0.0f, -1.0f, //4 (antes era el 8)
351         2.593003f, -0.085222f, 2.505755f, 0.49f, 0.76f, 0.0f, 0.0f, -1.0f, //5 (antes era el 6)
352         0.051152f, 2.399781f, 2.545043f, 0.49f, 0.99f, 0.0f, 0.0f, -1.0f, //6 (antes era el 3)
353         -2.646786f, -0.139047f, 2.493757f, 0.26f, 0.99f, 0.0f, 0.0f, -1.0f, //7 (antes era el 4)
354         // cara 3: cuadrado derecho (+X) -> casilla columna 3, renglon 1 (contando desde arriba a la izquierda)
355         2.632201f, -2.553892f, 0.007226f, 0.51f, 0.76f, -1.0f, -0.02f, 0.01f, //8 (antes era el 13
356         2.534944f, -0.061146f, -2.580280f, 0.74f, 0.76f, -1.0f, -0.02f, 0.01f, //9 (antes era el 9)

```

Cara 1 y 2 del holocrón.

```

416     MeshModel* holocron = new MeshModel();
417     //antes decia 120 y eran 128 floats (16x8), el vertice 15 se quedaba fuera del buffer
418     //ahora que lo cuenta el compilador: sizeof(arreglo completo) / sizeof(un elemento)
419     holocron->CreateMeshModel(holocron_vertices, holocron_indices, sizeof(holocron_vertices) / sizeof(GLfloat),
420     meshListModel.push_back(holocron);
421
422 }

```

```

444         //s holocronTexture = Texture("Textures/holocronunwrap.png");
445         holocronTexture = Texture("Textures/holocron_jedi.png");
446         holocronTexture.LoadTextureA();

```

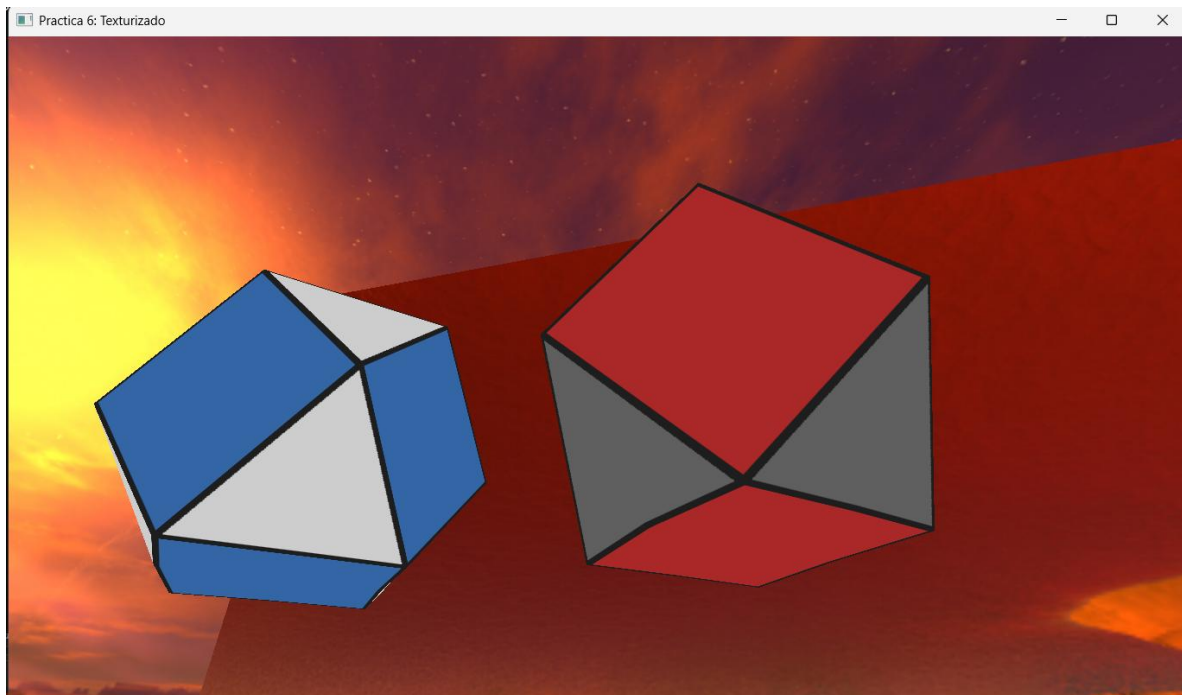
Carga de la textura

```

548
549     //Holocrones
550     //s color = glm::vec3(0.0f, 1.0f, 0.0f); //color que multiplica a la información de color de la textura
551     color = glm::vec3(1.0f, 1.0f, 1.0f);
552     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
553     model = glm::mat4(1.0);
554     //s model = glm::translate(model, glm::vec3(-8.5f, 4.5f, 0.0f));
555     model = glm::translate(model, glm::vec3(-4.5f, 2.5f, 4.0f));
556     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
557     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
558     holocronTexture.UseTexture();
559     meshListModel[5]->RenderMeshModel();

```

Dibujo del holocrón



Ejercicio 2.

```

448
449     Holocron_M = Model();
450     //s Holocron_M.LoadModel("Models/holocron_simple.obj");
451     Holocron_M.LoadModel("Models/holocron_texturizado.obj");
452     Avion_M = Model();
453     Avion_M.LoadModel("Models/avion_texturizado.obj");

```

Se carga el modelo

Dibujo del modelo

```

561     //s Ejercicio
562     color = glm::vec3(1.0f, 1.0f, 1.0f); //color blanco, multiplica a la i
563     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
564     model = glm::mat4(1.0);
565     model = glm::translate(model, glm::vec3(-4.5f, 2.5f, 0.0f));
566     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
567     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
568     Holocron_M.RenderModel();

```

El mismo holocrón, con las mismas coordenadas UV, se importó en Blender. Se le creó un material y en Base Color se colocó la imagen como Image Texture. Se exportó como OBJ con su archivo MTL, usando Path Mode: Strip para que el MTL guarde solo el nombre de la imagen, y se cargó en OpenGL con la clase Model.

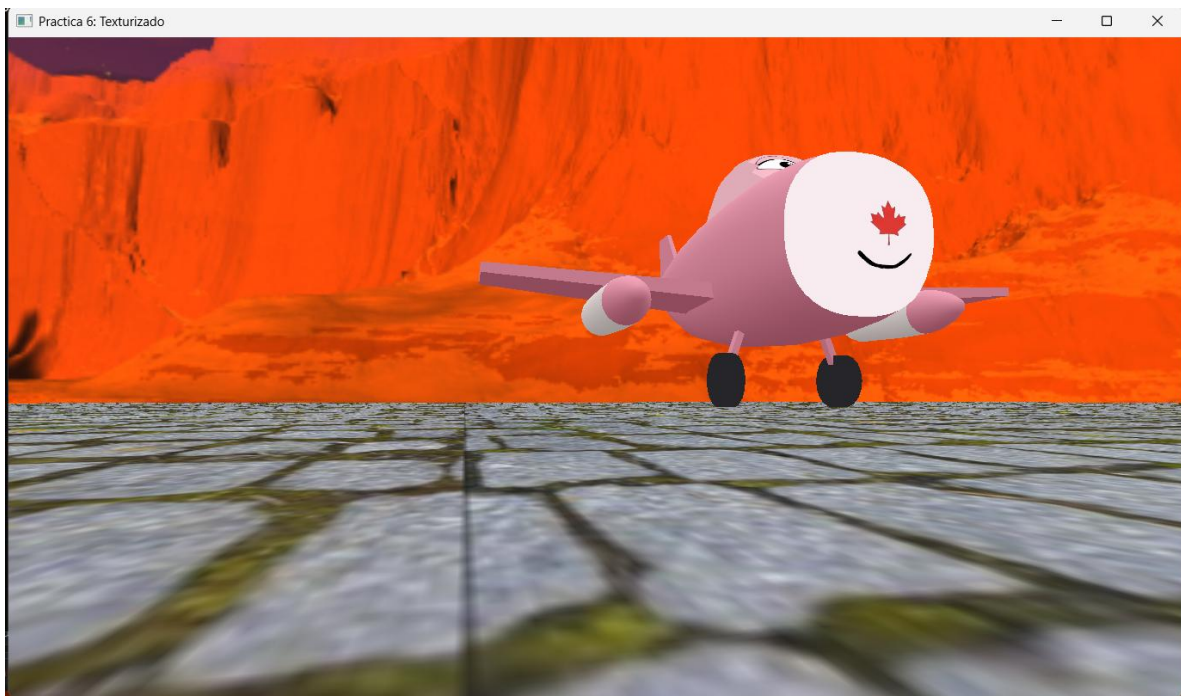
Ejercicio 3

```

452     Avion_M = Model();
453     Avion_M.LoadModel("Models/avion_texturizado.obj");
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471     //s Ejercicio 3: avion con la cara del personaje
472     color = glm::vec3(1.0f, 1.0f, 1.0f);
473     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
474     model = glm::mat4(1.0);
475     model = glm::translate(model, glm::vec3(3.5f, 1.5f, -7.0f));
476     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
477     Avion_M.RenderModel();

```

Se usó un modelo de avión generado con un script, con sus coordenadas UV separadas por zonas: vidrio frontal, frente y alas. En GIMP, sobre la plantilla de textura, se pegaron en la zona del vidrio los ojos recortados de la imagen del previo. En el frente se puso una hoja de maple como nariz y se dibujó la sonrisa, y en las alas se colocaron hojas de maple, el símbolo de Canadá que trae el personaje. La imagen se exportó en PNG de 1024×1024, en Blender se le asignó al avión y se exportó como OBJ con su MTL para cargarlo con la clase Model.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Faltaba la carpeta Textures/Skybox con sus imágenes. Sin ellas, el skybox nunca creaba su cubo y al dibujarlo usaba un puntero basura. Se resolvió copiando la carpeta Skybox.

El holocrón se veía de un solo color. Todas las S,T valían 0, así que todo el holocrón leía el mismo pixel de la imagen. Se resolvió asignando S,T por cara y repitiendo los vértices.

No se veía el holocrón de código. Desde la cámara quedaba justo detrás del holocrón importado. Se movió a (-4.5, 2.5, 4.0).

El avión no salía con su textura en OpenGL, aunque en Blender sí. El MTL pedía avion_plantilla.png y esa imagen no estaba en Textures, que es donde la busca Model.cpp. Se resolvió copiando la imagen a esa carpeta.

En GIMP se presentaron muchos problemas sobre cómo manejar capas.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Ejercicio 1: fue el más complejo. Hubo que entender que un vértice solo guarda un par S,T, por qué hay que repetir vértices y cómo repartir la imagen entre las caras.

Ejercicio 2: fue sencillo una vez que se sabe asignar el material y exportar. El detalle estuvo en que el MTL guarde bien el nombre de la imagen.

Ejercicio 3: complejidad media. Lo más tardado fue preparar la textura en GIMP: recortar, escalar y acomodar cada parte en su zona.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Me hubiera gustado que en clase fuéramos un poco más lento para poderle seguir el paso con GIMP.

c. Conclusión.

En esta práctica se vio que texturizar consiste en asociar cada vértice con un punto de la imagen mediante las coordenadas S,T. Hacerlo por código obliga a entender cómo se reparte la imagen entre las caras y por qué se repiten vértices. En Blender ese mapeo se hace de forma visual y se exporta junto con el modelo en el OBJ y el MTL. También quedó claro que la imagen debe estar donde el programa la busca y con el nombre correcto, y que un editor como GIMP sirve para preparar y acomodar las imágenes antes de usarlas como textura.

1. Bibliografía en formato APA

Assimp Team. (s.f.). Open Asset Import Library (assimp) [Software]. GitHub. <https://github.com/assimp/assimp>

Barrett, S. (s.f.). stb: Single-file public domain libraries for C/C++ [Software]. GitHub. <https://github.com/nothings/stb>

Blender Foundation. (s.f.). Manual de Blender. Recuperado el 8 de octubre de 2026, de <https://docs.blender.org/manual/es/latest/>

de Vries, J. (s.f.). Textures. LearnOpenGL. Recuperado el 8 de octubre de 2026, de <https://learnopengl.com/Getting-started/Textures>

Facultad de Ingeniería, UNAM. (2026). Manual de prácticas del Laboratorio de Computación Gráfica e Interacción Humano Computadora. Práctica 06: Texturizado. Universidad Nacional Autónoma de México.

The GIMP Documentation Team. (s.f.). Manual de usuario de GIMP. Recuperado el 8 de octubre de 2026, de <https://docs.gimp.org/>